
CIENCIA DE DATOS / Unidad 2 / 2.2

Manipulación y análisis de datos

El núcleo práctico de todo flujo de trabajo en datos: representar información en arreglos numéricos con **NumPy**, explorarla y limpiarla en tablas con **Pandas**, y comunicarla visualmente con **Matplotlib**.

2.2.1 NumPy para cómputo numérico

2.2.2 Pandas y análisis exploratorio de datos (EDA)

2.2.3 Visualización de datos con Matplotlib

2.2 Manipulación y análisis de datos

Los datos casi nunca llegan listos para usarse. Vienen en archivos con valores faltantes, tipos incorrectos, filas duplicadas y formatos inconsistentes. Antes de poder entrenar un modelo, calcular un indicador o tomar una decisión, hace falta **transformar esos datos crudos en información comprensible**. A ese trabajo —cargar, limpiar, transformar, resumir y visualizar— lo llamamos manipulación y análisis de datos, y suele consumir la mayor parte del tiempo de un proyecto real.

En el ecosistema de Python este trabajo se apoya en tres bibliotecas que se complementan y trabajan sobre la misma base. Conviene verlas como una **cadena**:

- **NumPy** — provee el arreglo numérico eficiente (`ndarray`), la estructura de datos sobre la que se construye todo lo demás.
- **Pandas** — provee tablas etiquetadas (`DataFrame`) para cargar, limpiar y explorar datos con nombres de columna, como una hoja de cálculo programable.
- **Matplotlib** — provee las herramientas para convertir esas tablas y arreglos en gráficos que revelan patrones que los números por sí solos ocultan.

NOTA

Convención de importación. Casi todo el mundo importa estas bibliotecas con los mismos alias. Los usaremos en todo el documento:

```
In [1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2.2.1 NumPy para cómputo numérico

NumPy (*Numerical Python*) es la biblioteca fundamental para el cómputo numérico. Su aporte central es un único tipo de objeto: el arreglo n-dimensional o `ndarray`, una rejilla de valores del mismo tipo que permite operar sobre millones de números de forma compacta y rapidísima.

DEFINICIÓN

ndarray. Estructura de datos de NumPy que almacena una colección de elementos del mismo tipo en memoria contigua. Se caracteriza por su *forma* (`shape`), el número de dimensiones (`ndim`) y el tipo de sus elementos (`dtype`).

¿Por qué no usar simplemente listas de Python?

Una lista de Python puede guardar números, pero es lenta para el cálculo masivo: cada elemento es un objeto independiente y toda operación exige un bucle interpretado. NumPy resuelve esto con dos ideas: guarda los datos en un bloque de memoria contiguo y ejecuta las operaciones en código C compilado, aplicándolas a todo el arreglo de una sola vez. A esto último se le llama **vectorización**.

```
In [2]:
# Sumar dos secuencias de un millón de números
lista_a = list(range(1_000_000))
arr_a    = np.arange(1_000_000)

# Con listas: hace falta un bucle explícito
resultado = [x + y for x, y in zip(lista_a, lista_a)]

# Con NumPy: una sola expresión, sin bucle
resultado = arr_a + arr_a    # ~50-100x más rápido
```

CLAVE

La regla mental de NumPy: si te encuentras escribiendo un bucle for para recorrer números, casi siempre existe una operación vectorizada que lo reemplaza —más corta, más legible y mucho más veloz.

Crear arreglos

Hay varias formas de construir un ndarray según lo que se necesite:

```
In [3]:
np.array([1, 2, 3, 4])      # desde una lista
np.zeros((2, 3))           # matriz 2x3 de ceros
np.ones(5)                 # vector de cinco unos
np.arange(0, 10, 2)         # [0 2 4 6 8] (inicio, fin, paso)
np.linspace(0, 1, 5)        # 5 valores igualmente espaciados en [0,1]
```

```
Out[3]:
linspace -> [0.   0.25 0.5  0.75 1.   ]
```

Forma, dimensión y tipo

Todo arreglo describe su estructura mediante tres atributos que conviene consultar siempre:

```
In [4]:
a = np.array([[1, 2, 3],
              [4, 5, 6]])
a.shape      # (2, 3) -> 2 filas, 3 columnas
a.ndim       # 2      -> es bidimensional
a.dtype      # int64   -> tipo de los elementos
a.size       # 6       -> total de elementos
```

Indexado, rebanado y máscaras booleanas

Se accede a los elementos con corchetes. Además del indexado clásico, NumPy permite **rebanado** (*slicing*) y —muy potente— el **filtrado por condición**, que selecciona todos los elementos que cumplen un criterio:

```
In [5]:
x = np.array([10, 20, 30, 40, 50])
x[0]      # 10      (primer elemento)
x[-1]     # 50      (último elemento)
x[1:4]    # [20 30 40] (desde el índice 1 hasta el 3)
```

```
# Máscara booleana: "dame los mayores a 25"
x[x > 25]      # [30 40 50]
```

EJEMPLO

Filtrar sin bucles. La expresión `x > 25` produce un arreglo de `True/False` del mismo tamaño; al usarlo como índice, NumPy devuelve solo las posiciones `True`. Es la base del filtrado de datos en Pandas.

Operaciones y reducciones por eje

Las operaciones aritméticas se aplican elemento a elemento. Las **reducciones** (suma, media, máximo...) colapsan el arreglo a un resumen; en 2D podemos elegir el **eje**: `axis=0` opera por columnas y `axis=1` por filas.

```
In [6]:
m = np.array([[1, 2, 3],
              [4, 5, 6]])
m.sum()           # 21 (todo el arreglo)
m.mean(axis=0)    # [2.5 3.5 4.5] (media por columna)
m.sum(axis=1)     # [ 6 15] (suma por fila)
m.max()           # 6
```

Broadcasting

DEFINICIÓN

Broadcasting. Mecanismo por el cual NumPy opera automáticamente entre arreglos de formas distintas pero compatibles, "estirando" el más pequeño para que coincida con el más grande, sin copiar datos.

Gracias al broadcasting, escalar o centrar datos es una sola línea. Por ejemplo, restar la media de cada columna a toda la matriz:

```
In [7]:
datos = np.array([[10., 20.],
                  [30., 40.],
                  [50., 60.]])
media = datos.mean(axis=0) # [30. 40.] -> una fila
centrado = datos - media  # se resta a CADA fila
```

```
Out[7]:
array([[ -20.,  -20.],
       [  0.,   0.],
       [ 20.,  20.]])
```

Ejemplo integrador: normalización

Una tarea habitual antes de entrenar modelos es la **normalización min-max**, que reescala los valores al rango `[0, 1]` con la fórmula:

$$x_{\text{norm}} = (x - \min(x)) / (\max(x) - \min(x))$$

```
In [8]:
notas = np.array([50, 65, 80, 92, 100])
x_min, x_max = notas.min(), notas.max()
normalizadas = (notas - x_min) / (x_max - x_min)
print(np.round(normalizadas, 2))
```

```
Out[8]:
[0.  0.3  0.6  0.84 1.  ]
```

NOTA

Toda la operación se resolvió sin un solo bucle: `.min()` y `.max()` son reducciones, y la resta y división se aplican por broadcasting a los cinco valores a la vez. Ese es el estilo idiomático de NumPy.

2.2.2 Pandas y análisis exploratorio de datos (EDA)

Los datos reales rara vez son un simple vector de números: son **tablas** con columnas de distinto tipo —nombres, fechas, categorías, cantidades— y con un significado por columna. Pandas construye sobre NumPy dos estructuras pensadas para esto: la **Series** (una columna etiquetada) y el **DataFrame** (una tabla completa de filas y columnas con nombre).

DEFINICIÓN

Series. Arreglo unidimensional con un índice (etiqueta) para cada valor. Es, en esencia, una columna.

DataFrame. Estructura bidimensional de columnas etiquetadas, donde cada columna es una Series y puede tener su propio tipo. Es la tabla con la que trabajaremos casi siempre.

Cargar e inspeccionar datos

El punto de partida típico es leer un archivo CSV. Usaremos como ejemplo un pequeño conjunto de ventas. Lo primero, siempre, es **mirar** los datos: sus primeras filas, su tamaño, sus tipos y un resumen estadístico.

```
In [9]:
df = pd.read_csv('ventas.csv')
df.head()      # primeras 5 filas
df.shape       # (filas, columnas)
df.info()      # tipos y valores no nulos por columna
df.describe()  # resumen estadístico de las columnas numéricas
```

Así se ven las primeras filas del conjunto (`df.head()`):

fecha	region	producto	unidades	precio
2026-01-03	Norte	Teclado	12	90
2026-01-03	Sur	Mouse	30	45
2026-01-04	Norte	Monitor	5	NaN
2026-01-04	Este	Mouse	18	45
2026-01-05	Sur	Teclado	9	90

ATENCIÓN

Fíjate en el **NaN** de la columna precio. Es un valor faltante (*Not a Number*), y detectarlo temprano es parte esencial del análisis exploratorio.

Seleccionar y filtrar

Pandas ofrece dos accesores clave: `.loc` selecciona por *etiqueta* (nombres de columna o índice) e `.iloc` por *posición* numérica. El filtrado por condición funciona igual que las máscaras de NumPy:

```
In [10]:
df['region']      # una columna (Series)
```

```
df[['producto', 'unidades']]      # varias columnas (DataFrame)
df.iloc[0]                        # primera fila por posición
df.loc[df['region'] == 'Norte']    # filas de la región Norte
df[df['unidades'] > 15]           # ventas de más de 15 unidades
```

Limpiar los datos

Antes de analizar hay que resolver los problemas más comunes: valores faltantes, duplicados y tipos incorrectos.

```
In [11]:
df.isna().sum()                  # cuántos faltantes hay por columna
df['precio'].fillna(df['precio'].median(), inplace=True)
df.dropna()                      # (alternativa) descartar filas con faltantes
df.drop_duplicates()             # eliminar filas repetidas
df['fecha'] = pd.to_datetime(df['fecha']) # corregir el tipo
```

NOTA

Imputar vs. descartar. Rellenar el faltante con un valor razonable (la mediana, aquí) conserva la fila y suele ser preferible a `dropna()` cuando los datos son escasos. La decisión depende del contexto: no hay una regla única.

Transformar y agrupar

El patrón más poderoso de Pandas es **agrupar y resumir** con `groupby`: divide la tabla en grupos según una columna, aplica un cálculo a cada grupo y combina los resultados. Primero creamos una columna derivada:

```
In [12]:
df['ingreso'] = df['unidades'] * df['precio']

# Ingreso total por región
df.groupby('region')['ingreso'].sum().sort_values(ascending=False)

Out[12]:
region
Norte    1080.0
Sur      1170.0
Este      810.0
Name: ingreso, dtype: float64
```

¿Qué es el Análisis Exploratorio de Datos (EDA)?

DEFINICIÓN

EDA (Exploratory Data Analysis). Fase inicial en la que se examina un conjunto de datos para comprender su estructura, detectar errores y valores atípicos, descubrir patrones y relaciones, y formular hipótesis —antes de aplicar modelos. Combina estadística descriptiva y visualización.

El EDA no es un paso mecánico, pero sí sigue un recorrido reconocible. Una guía práctica:

- **Panorama general:** tamaño, columnas, tipos y memoria (`shape`, `info`).

- **Calidad:** faltantes, duplicados y valores imposibles (edades negativas, fechas futuras).
- **Análisis univariado:** distribución de cada variable por separado (media, mediana, dispersión, histogramas).
- **Análisis bivariado:** relaciones entre pares de variables (correlaciones, dispersión, comparación entre grupos).
- **Síntesis:** anotar hallazgos y preguntas que guiarán la siguiente etapa.

Un resumen numérico rápido con `describe()` ya revela mucho —rango, media, dispersión— y es el primer gesto de casi cualquier EDA:

```
In [13]:  
df.describe()
```

	unidades	precio	ingreso
count	5.0	5.0	5.0
mean	14.8	66.0	612.0
std	9.7	24.6	399.9
min	5.0	45.0	225.0
max	30.0	90.0	1350.0

EJEMPLO

Lectura del resumen. La desviación estándar de ingreso (≈ 400) es casi tan grande como la media (612): los ingresos varían mucho entre ventas. Ese tipo de observación es exactamente lo que el EDA busca antes de modelar.

2.2.3 Visualización de datos con Matplotlib

Los números resumen; los gráficos **revelan**. Una tendencia, un valor atípico o una relación entre variables suelen saltar a la vista en una figura mucho antes que en una tabla. Matplotlib es la biblioteca base para crear estos gráficos en Python, y da control total sobre cada detalle de la figura.

La anatomía de una figura

Entender dos objetos evita casi toda la confusión inicial. La **Figure** es el lienzo completo —la ventana o imagen entera—. Dentro viven uno o más **Axes**, que son las regiones de dibujo (cada gráfico con sus ejes X e Y). Un Axis no es "un eje": es un sistema de coordenadas completo donde se traza.

CLAVE

Dos formas de usar Matplotlib. La interfaz pyplot (`plt.plot(...)`) es rápida para gráficos simples. La interfaz **orientada a objetos** (`fig, ax = plt.subplots()`) da más control y es la recomendada para figuras serias. Usaremos sobre todo la segunda.

Gráfico de líneas: tendencias en el tiempo

Ideal para mostrar cómo cambia una magnitud a lo largo de una secuencia, típicamente el tiempo.

```
In [14]:
dias    = [1, 2, 3, 4, 5]
ingreso = [225, 1350, 450, 810, 810]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(dias, ingreso, marker='o', color='#4F46E5')
ax.set_title('Ingreso diario')
ax.set_xlabel('Día')
ax.set_ylabel('Ingreso (Bs)')
plt.show()
```

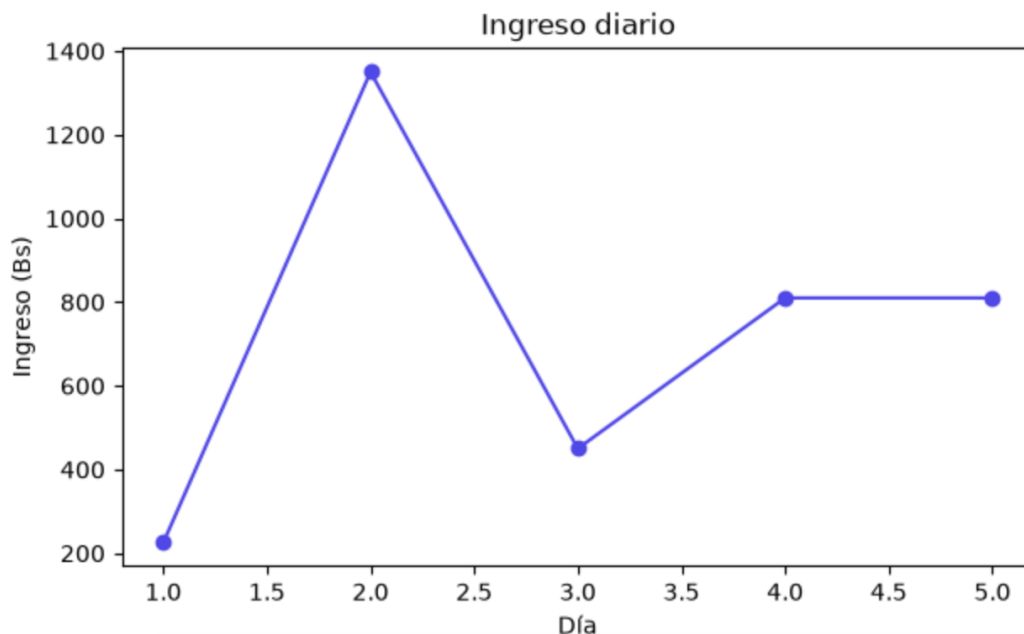


Gráfico de barras: comparar categorías

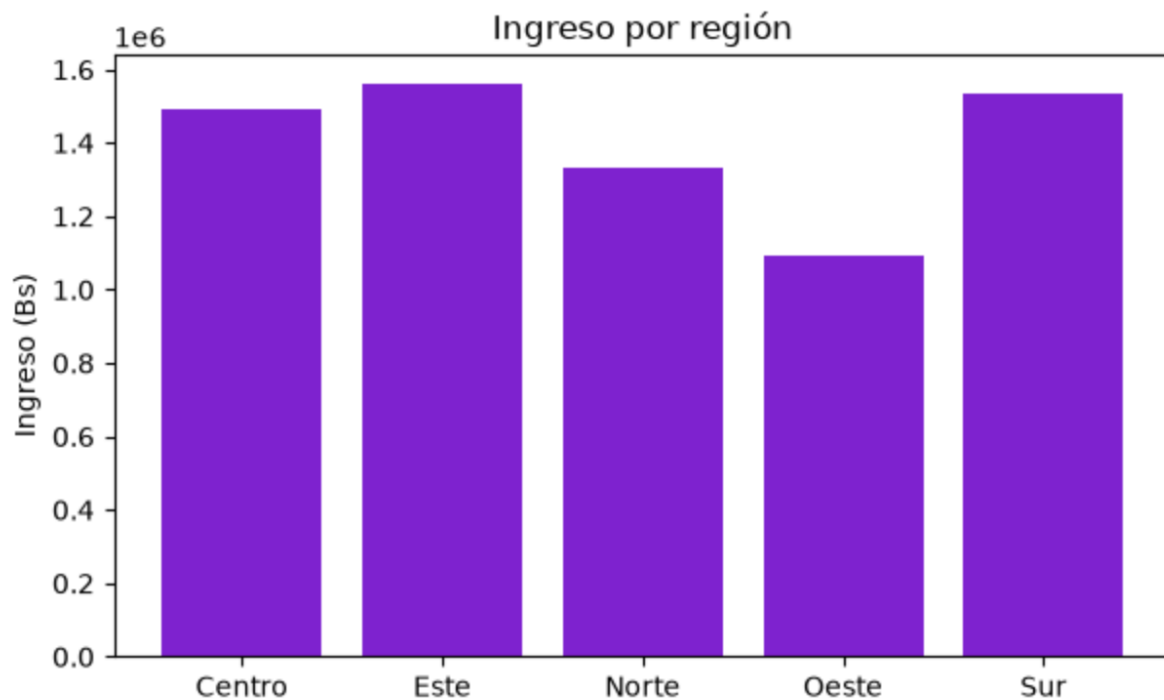
El más claro para comparar una cantidad entre grupos discretos, como el ingreso por región.

```
In [15]:
resumen = df.groupby('region')['ingreso'].sum()

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(resumen.index, resumen.values, color='#7E22CE')
ax.set_title('Ingreso por región')
ax.set_ylabel('Ingreso (Bs)')
plt.show()
```

NOTA

Pandas y Matplotlib se integran directamente: `resumen.plot(kind='bar')` produce el mismo gráfico en una línea. Pandas llama a Matplotlib por debajo.

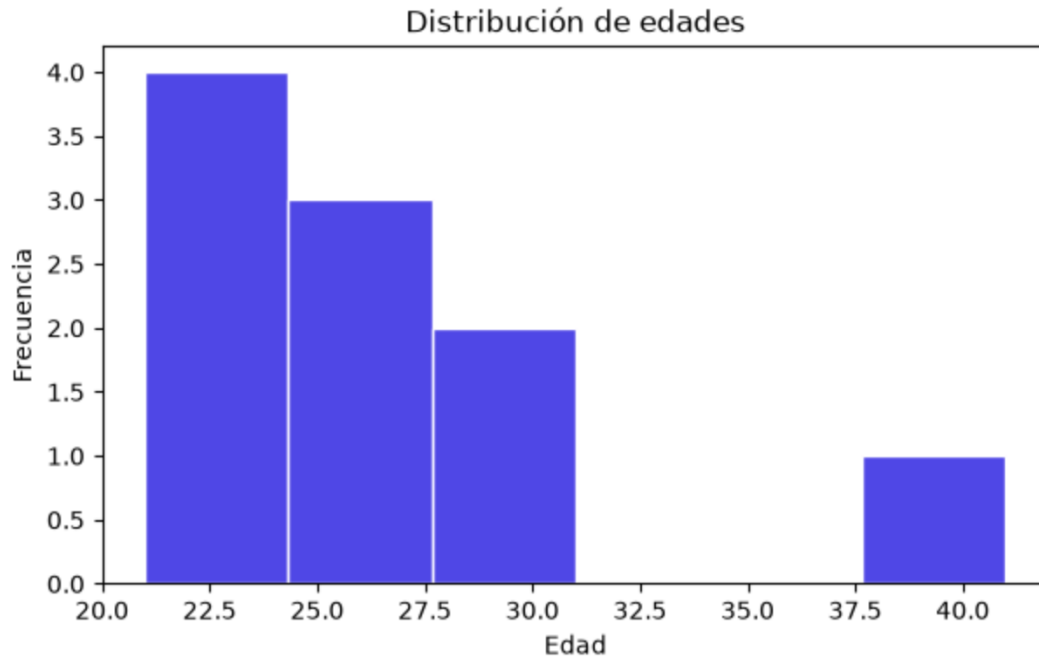


Histograma: la forma de una distribución

Fundamental en el EDA univariado: agrupa los valores en intervalos (bins) y cuenta cuántos caen en cada uno, mostrando dónde se concentran los datos, si son simétricos y si hay valores extremos.

```
In [16]:
edades = np.array([21, 22, 22, 23, 25, 25, 25, 28, 30, 41])

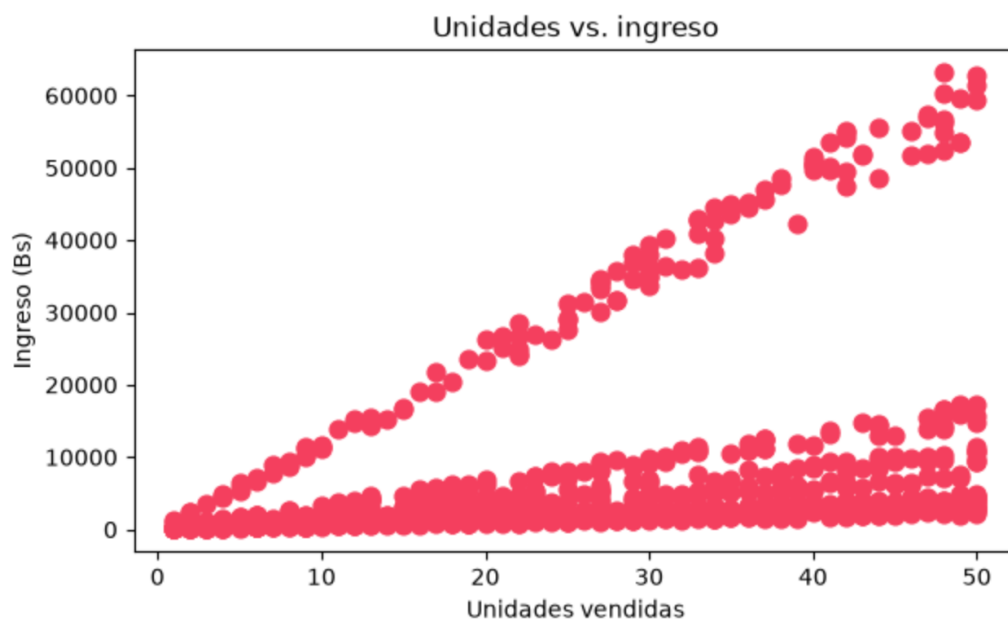
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(edades, bins=6, color='#4F46E5', edgecolor='white')
ax.set_title('Distribución de edades')
ax.set_xlabel('Edad'); ax.set_ylabel('Frecuencia')
plt.show()
```



Dispersión: relación entre dos variables

El *scatter plot* es la herramienta del EDA bivariado: cada punto es una observación con sus valores en X e Y, y su nube revela si dos variables se relacionan.

```
In [17]:  
fig, ax = plt.subplots(figsize=(7, 4))  
ax.scatter(df['unidades'], df['ingreso'], color='#F43F5E', s=60)  
ax.set_xlabel('Unidades vendidas')  
ax.set_ylabel('Ingreso (Bs)')  
ax.set_title('Unidades vs. ingreso')  
plt.show()
```



Varios gráficos en una figura (subplots)

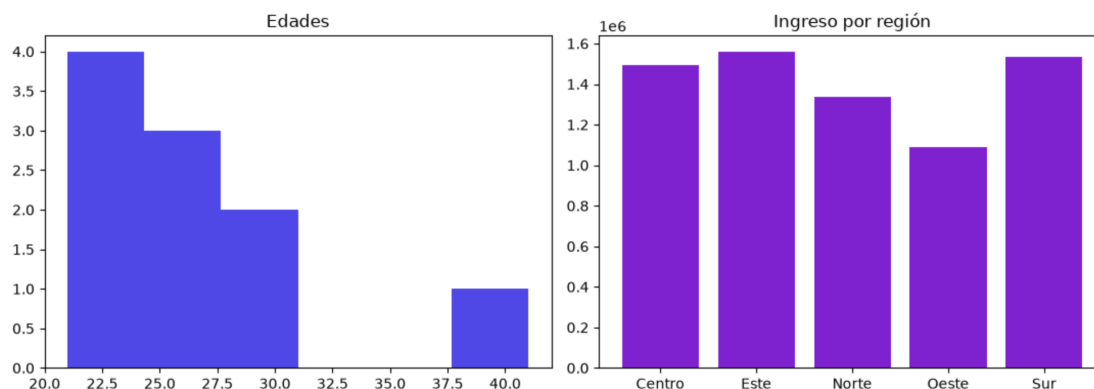
Con `plt.subplots(filas, columnas)` se crea una cuadrícula de ejes. `ax` pasa a ser un arreglo, y cada posición recibe su propio gráfico:

```
In [18]:
fig, ax = plt.subplots(1, 2, figsize=(11, 4))

ax[0].hist(edades, bins=6, color='#4F46E5')
ax[0].set_title('Edades')

ax[1].bar(resumen.index, resumen.values, color='#7E22CE')
ax[1].set_title('Ingreso por región')

fig.tight_layout() # evita que se solapen los títulos
plt.show()
```



Personalizar y guardar

Un gráfico útil se explica solo: título, ejes rotulados, leyenda cuando hay varias series y, si va a un informe, guardado en alta resolución.

```
In [19]:
ax.legend(['Norte', 'Sur']) # leyenda cuando hay varias series
ax.grid(True, alpha=0.3)   # rejilla tenue de fondo
fig.savefig('grafico.png', dpi=300, bbox_inches='tight')
```

ATENCIÓN

Orden importa. Llama a `savefig()` antes de `plt.show()`: en algunos entornos `show()` vacía la figura y el archivo guardado saldría en blanco.

Cierre: el flujo completo

Las tres herramientas forman un ciclo natural de trabajo. Un caso típico recorre: **cargar y limpiar** con Pandas → **calcular y transformar** apoyándose en NumPy → **resumir** con estadística descriptiva → **visualizar** con Matplotlib para comunicar los hallazgos. Dominar este ciclo es dominar la base práctica de la ciencia de datos; todo lo que viene después —modelado, machine learning— se construye sobre él.